הרצאה 5: יסודות התכנון דינמי

(Dynamic Programming)

קיימת בעיות חישוביות רבות שאפשר לפתור אותן באמצעות אלגוריתם רקורסיבי בסיבוכיות מעריכית.

תכנון דינמי הוא פרדיגמה (שיטת חשיבה) המאפשרת פתרון יעיל לחלק מן הבעיות האלה.

בתחילת ההרצאה נציג בעיה פשוטה ונפתור אותה תוך שימוש בתכנון דינמי. לאחר מכן נציג את עיקרי השיטה באופן כללי.

נסיים את ההרצאה בהצגת עוד כמה דוגמאות.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה: $P1$

בעיה $P1$ מוצגת באיור 5.1 .

קלט

מערך A שאיבריו מספרים חיוביים.

פלט

תת מערך (לא רציף) שסכום איבריו מרבי, ואשר אינו מכיל שני איברים סמוכים.

איור 5.1: הבעיה $P1$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמאות להבהרת *P1*

נתבונן בדוגמאות הבאות המבהירות את הבעיה:

הקלט

9	4	**5**

הפתרון: 14

הקלט

6	**12**	3

הפתרון: 12

הקלט

9	**17**	6	5	**12**

הפתרון: 29

הקלט

9	5	3	**13**

הפתרון: 22

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מסקנות

מהדוגמאות אפשר להסיק:

1. מספר האיברים הרצופים שאינם כלולים בתת המערך האופטימלי הוא לכל היותר 2.
2. אחד משני האיברים האחרונים במערך הקלט נכלל תמיד במערך הפלט.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 5.2

יהי A מערך מספרים. אם Opt הוא פתרון ל-$P1$ נאמר ש-Opt הוא **תת מערך אופטימלי** של A.

מסקנה 5.3

יהי A מערך מספרים ויהי Opt תת מערך אופטימלי של . אזי:

1. המערך Opt מכיל לפחות איבר אחד מכל שלישית איברים רצופים.
2. המערך Opt מכיל את $A[n-1]$ או את $A[n]$ אך לא את שניהם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת סעיף 1

נניח בשלילה כי האינדקס i,

$1 \le i \le n-2$ מקיים: תת המערך Opt

לא מכיל את $A[i]$, $A[i+1]$, וגם את

$A[i+2]$. נתון כי $A[i+1] > 0$, צרוף

$A[i+1]$ ל-Opt מניב תת מערך

המקיים את דרישות $P1$ שסכום

איבריו גדול מסכום איברי Opt

- סתירה לאופטימליות Opt.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת סעיף 2

יהי A מערך מספרים ויהי Opt תת מערך אופטימלי של A. מסעיף 1 נובע:

- אם $A[n-2]$ לא נכלל ב-Opt אז האיבר הגדול מבין $A[n-1]$ ו-$A[n]$ נכלל בו.
- אם $A[n-2]$ נכלל ב-Opt, ברור שגם $A[n]$ נכלל בו.

מש"ל

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם $P1S$

כשלב ביניים לפתרון הבעיה נציג את אלגוריתם $P1S$ המחשב את סכום האיברים של כל תת מערך אופטימלי של מערך הקלט A. האלגוריתם משתמש בשיטה הרקורסיבית $P1R$ שנציג כעת.

השיטה $P1R$ מקבלת כקלט מערך מספרים A ואינדקס $1 \le i \le n$ ומחזירה את סכום איברי כל תת מערך אופטימלי של $A[1...i]$ תחת האילוץ ש-$A[i]$ נכלל בפתרון.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

לדוגמה:

עבור המערך $A=[3,2,4,3]$

$$P1R(A,3)=A[1]+A[3]=7.$$

$$P1R(A,4)=A[1]+A[4]=6.$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

```
P1S(A)
    return(Max(P1R(A,n),P1R(A,n-1)))
-----------------------------------------
P1R(A,i)
    if ((i=1)||(i=2)) return(A[i]) end if
    if (i=3) return(A[3]+A[1]) end if
    return(A[i]+max(P1R(A,i-2),
                    P1R(A,i-3)))
end(P1R)
```

איור 5.4: אלגוריתם *P1S* והשיטה *P1R*

תיאור P1R

השיטה $P1R$ היא רקורסיבית. מקרי הקצה הם $i=1,2,3$ והם נבדקים בשורה הראשונה והשנייה. ההוראה

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בשורה השלישית מחשבת את הפלט עבור $i > 3$ כדי להבין הוראה זו נשים לב שלפי סעיף 1, בלמה 5.2, לפחות אחד מן האיברים: $A[i-3]$, $A[i-2]$, או $A[i-1]$ נכלל בפתרון.

נתון ש-$A[i]$ נכלל בפתרון, ולכן $A[i-1]$ לא נכלל בו. כמו כן, לא ייתכן שגם $A[i-3]$, וגם $A[i-2]$ יכללו, ומכאן: הסכום המרבי יתקבל על ידי הביטוי

$$P1r(A,i) = A[i] + \max\begin{Bmatrix} P1R(A,i-2), \\ P1R(A,i-3) \end{Bmatrix}$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור אלגוריתם $P1S$

לפי סעיף 2 בלמה 5.2 כל תת-מערך אופטימלי של A מכיל או את $A[n-1]$ או את $A[n]$. אלגוריתם $P1S$ מבצע את הקריאות $P1R(A,n-1)$ ו-$P1R(A,n)$ ומחזיר את הגדול מבין שני הערכים המוחזרים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 5.5: נכונות $P1S$

1. השיטה הרקורסיבית $P1R(A,i)$ המבוצעת עם מערך קלט A מחזירה את סכומם של איברי תת מערך אופטימלי של הרישה $A[1,\ldots,i]$ תחת האילוץ שהאיבר $A[i]$ שייך לסכום.
2. אלגוריתם $P1S$ המבוצע עם מערך הקלט A מחזיר את סכומם של איברי כל תת מערך אופטימלי של A..

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת סעיף 1

נוכיח את הטענה באינדוקציה על מספר איברי הרישה $A[1,\ldots,i]$.

תת מערך רציף של מערך A,לדוגמה $A[1,\ldots,6]$ הוא **רישה** של A.

בסיס ($i=1,2,3$)

נכונות האלגוריתם עבור מקרי הקצה נובעת ישירות מהגדרת השיטה $P1R$.

צעד

נניח כי הקריאה $P1R(A,j)$ מחזירה פתרון נכון לכל $1 \le j < i$, ונוכיח זאת לגבי i.

נתון ש-$A[i]$ נכלל בסכום. לפי הגדרת הבעיה, $A[i-1]$ לא נכלל בסכום.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מסעיף 1 בלמה 5.2 נובע כי האיבר $A[i-2]$ או האיבר $A[i-3]$ ייכלל בסכום.

פונקצית ה-max בשורה האחרונה של הקוד בוחרת את האפשרות שתרומתה לסכום מרבית.

<u>הוכחת סעיף 2</u>

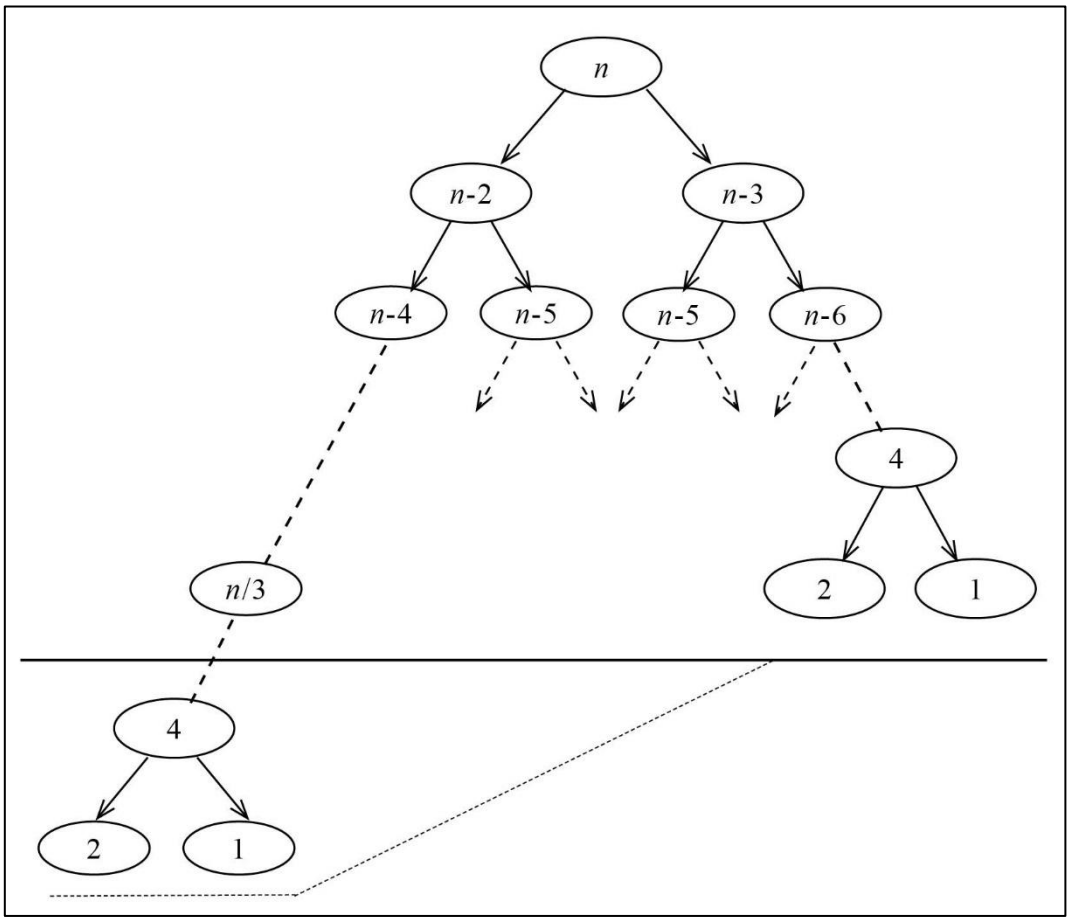
לפי סעיף 1 הקריאות $P1R(A,n-1)$ ו-$P1R(A,n)$ מחזירות פתרונים אופטימליים. לפי למה 5.2 הפתרון האופטימלי לבעיה $P1$ חייב להכיל את $A[n-1]$ או את $A[n]$. האלגוריתם מחזיר את $\max\{P1R(A,n-1),P1R(A,n)\}$.

מש"ל

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות *P1R* ו-*P1S*

עץ הרקורסיה של *P1R* מתואר באיור 5.6 . אפשר לראות כי תת העץ מעל לקו המאוזן הוא עץ בינארי מאוזן בגובה $n/3$.



איור 5.6: עץ הרקורסיה של *PIR*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בתת העץ הזה יש (בערך) $2^{n/3}$ צמתים, וכל צומת מייצג קריאה רקורסיבית, כלומר: מספר הקריאות הרקורסיביות הוא **מעריכי** (אקספוננציאלי), מכאן: סיבוכיות $P1S$ היא מעריכית.

<u>תזכורת</u>

אנו נמנעים משימוש באלגוריתמים מעריכיים ככל האפשר.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגורם לסיבוכיות הגבוהה

השיטה $P1R$ מופעלת עם הארגומנטים A ו-i כאשר הערכים האפשריים של i הם המספרים $1,2,...,n$ ומספרם הוא בדיוק n.

מספר הקריאות ל-$P1R$ הוא מעריכי.

מכאן נובע: $P1R$ מתבצעת פעמים רבות עם אותם הארגומנטים.

לדוגמה

אפשר לראות שברמה השלישית של עץ הרקורסיה יש שתי קריאות עם הארגומנט $i = n - 5$. כל קריאה כזו משרה עץ שמספר הקריאות בו הוא מעריכי.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תכנות (או תכנון) דינמי

שיטת התכנות הדינמי מחשבת את התוצאות של כל אחת מן הקריאות הרקורסיביות ללא שימוש בהן.

אלגוריתם $P1D$ הפותר את בעיה $P1$ בתכנות דינמי מוצג באיור 5.7.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם בתכנון דינמי - *P1D*

```
P1D(A)
    B[1] ← A[1]
    B[2] ← A[2];
    B[3] ← B[1] + A[3];
    for i ← 4 to n
        B[i] ← A[i] +
            + max(B[i − 3], B[i − 2])
    endfor
    return(max(B[n], B[n − 1]))
end(P1D)
```

איור 5.7: אלגוריתם *P1D*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור האלגוריתם

אלגוריתם $P1D$ מחשב את **הערכים המוחזרים מכל הקריאות לשיטה $P1R$ ללא קריאה לשיטה עצמה.**

לכל $i=1,\ldots,n$ לאחר חישוב תוצאת הקריאה $P1R\left(A,i\right)$ האלגוריתם מאחסן את התוצאה שהתקבלה במערך B המשמש כ**טבלת מעקב (Look up Table)** ובה n איברים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

השוואת $P1r$ עם $P1D$

```
P1R(A,i)
    if ((i=1)||(i=2)) return(A[i]) end if
    if (i=3) return(A[3]+A[1]) end if
    return(A[i]+max(P1R(A,i-2),
                    P1R(A,i-3)))
end(i=3)
```

```
P1D(A)
    B[1] ← A[1];  B[2] ← A[2];
    B[3] ← A[1]+A[3];
    for i ← 4 to n
        B[i] ← A[i]+max(B[i-2],B[i-3])
    endfor
    return(max(B[n],B[n-1]))
end(P1D)
```

איור 5.8: השוואת $P1R$ עם $P1D$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה: חישוב איברי B

נניח ש-$P1D$ מתבצע עם הקלט המוצג באיור 5.8

A	5	2	3	8	4	6	9	8	7	5

איור 5.9: הקלט

$P1D$ מחשב את ערכי B בסדר אינדקסים עולה. שלושת האיברים הראשונים מחושבים על ידי שימוש בתנאי הקצה: $B[1]=5$, $B[2]=2$, $B[3]=3+5=8$.

כדי לחשב את $B[i]$, $4\le i\le n$, $P1D$ משתמש בחישוב זהה לחישוב בו משתמשת $P1R$, אך במקום לקרוא

© כל הזכויות שמורות לפרופסור עמוס ישראלי

ל-$P1R(A,i-2)$, ול-$P1R(A,i-3)$

$P1D$ משתמש בערכים המאוחסנים

ב-$B[i-2]$, ובו-$B[i-3]$.

המערכים A ו-B מוצגים באיור 5.10

האינדקס	1	2	3	4	5	6	7	8	9	10
A	5	2	3	8	4	6	9	8	7	5
B	5	2	8	13	12	19	22	27	29	32

<u>איור 5.10</u>

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 5.11: נכונות $P1D$

אלגוריתם $P1D$ מקבל כקלט מערך מספרים חיוביים, A, ומחזיר את סכום כל תת מערך אופטימלי של A.

הוכחה

$P1D$ הוא אלגוריתם איטרטיבי. הוכחת הנכונות היא באינדוקציה על מספר המעברים בלולאה. נקודת השמורה היא לפני סיום הלולאה והשמורה היא:

לכל j, $1 \le j \le i$, ערכו של $B[j]$ שווה לסכום איברי תת מערך אופטימלי המכיל את $A[j]$. (כלומר לערך המוחזר על ידי $P1R(A, j)$).

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הוכחת נכונות השמורה, ונכונות $P1D$,
זהה להוכחת הנכונות עבור $P1R$,
ו-$P1S$.

משי״ל

<u>שימו לב</u>

במקרה זה, כמו במקרים רבים של אלגוריתמים בתכנון דינמי, השמורה שווה להגדרת איברי מערך העזר.

<u>הסיבוכיות</u>

לינארית.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

חישוב תת מערך אופטימלי

כעת נחשב את (אחד מ) תת המערכים האופטימליים של מערך הקלט ולא רק את סכום האיברים שלו.

כדי לבצע זאת נגדיר מערך עזר נוסף, $Prev$, כאשר ב- $Prev[i]$, יאוחסן האינדקס של האיבר שהשתתף בחישוב $B[i]$. איבר זה הוא הגדול מבין $B[i-2]$, ו-$B[i-3]$. באיור 5.12, מוצג $P1DF$, כולל הקוד לחישוב מערך העזר $Prev$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

```
P1DF(A)
    B[1] ← A[1]; B[2] ← A[2];
    B[3] ← A[1] + A[3]; Prev[3] ← 1
    for i ← 4 to n
        if(B[i−3] > B[i−2])
            B[i] ← A[i] + B[i−3]
            Prev[i] ← (i−3)
        else
            B[i] ← A[i] + B[i−2]
            Prev[i] ← (i−2)
        end if
    end for
    if(B[n−1] > B[n]) mind ← (n−1)
      else mind ← n
    end if
    return CompOptIndArray(Prev, mind)
end(P1DF)
```

איור 5.12: השיטה *P1DF*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

באיור 5.13 מופיעים הקלט ומבני הנתונים שהאלגוריתם יוצר:

האינדקס	1	2	3	4	5	6	7	8	9	10
A	5	2	3	8	4	6	9	8	7	5
B	5	2	8	13	12	19	22	27	29	32
$Prev$	-	-	1	1	3	4	4	6	7	8

איור 5.10: מבנה הנתונים

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הקוד לחישוב מערך האינדקסים

```
CompOptIndArray(Prev, mind)
    ind ← mind
    j ← 1
    do
        RevIndSeries[j] ← ind ; j++
        ind ← Prev[ind]
    while(Prev[ind] > 0)
    return Reverse(RevIndArray)
end(CompOptArray)
```

השיטה *CompOptIndArray*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

בדיקה

אם נתבונן במבנה הנתונים, אפשר לראות כי הסכום

$B[10]=32$ התקבל על ידי

$$B[10]=A[10]+B[8]$$

אם נמשיך בדרך זו נקבל:

$$B[8]=A[8]+B[6]$$

$$B[6]=A[6]+B[4]$$

$$B[4]=B[1]+A[4]$$

$$B[10]=A[10]+A[8]+A[6]+A[4]+A[1]$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיכום הצעדים שביצענו בבנית $P1D$

1. פתרנו את $P1$ בעזרת השיטה הרקורסיבית $P1R$ אשר מחשבת את סכום תת המערך המרבי עבור תת המערך $A[1:i]$, כאשר נתון ש-$A[i]$ שייך לפתרון.
2. הגדרנו מערך עזר B כאשר $B[i] = P1R(A,i)$.
3. חישבנו את איברי מערך העזר.
4. חישבנו את סכום תת המערך המרבי עבור המערך A.
5. חישבנו את האינדקסים של האיברים של תת המערך המרבי.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מתכון לבניית אלגוריתם בתכנון דינמי

1. הציגו שיטה רקורסיבית R לפתרון הבעיה.
2. אם סיבוכיות R גבוהה מאוד, אך מספר הארגומנטים השונים זה מזה, אשר R מופעלת עליהם, נמוך יחסית, אז:
3. השתמשו בטבלת מעקב, חשבו את ערכי R על הארגומנטים השונים בעזרת טבלת המעקב, **וללא קריאה לשיטה** R עצמה.
4. השתמשו בטבלה שבניתם כדי לחשב את הפלט.

איור 5.11: מתכון לאלגוריתם בתכנון דינמי

© כל הזכויות שמורות לפרופסור עמוס ישראלי

שימו-לב

לאחר לימוד שיטת התכנון הדינמי ורכישת ניסיון בשיטה זו, אפשר לוותר על פיתוח אלגוריתם רקורסיבי:
כאשר מזהים בעיה הניתנת לפתרון על ידי אלגוריתם רקורסיבי, ומספר ערכי הארגומנטים של האלגוריתם קטן.
מגדירים ישירות את טבלת המעקב, ומפתחים את האלגוריתם תוך שימוש בטבלה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיכום

בהרצאה זו הצגנו את שיטת ה**תכנון הדינמי** אשר משמשת להורדת סיבוכיות של אלגוריתמים רקורסיביים בהם מתבצע מספר רב של קריאות רקורסיביות עם ארגומנטים זהים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי